

Day 18: Packages

- Creating packages
- Importing packages
- Java API packages

Today we are learning **Packages** in Java.

You already learned:

- Class & Object
- Constructor
- Inheritance
- Polymorphism
- Encapsulation

Now imagine your Java project becomes **very big**. You may have 100, 500, or even 1000 classes.

If all classes are kept in one place, finding and managing them becomes difficult.

👉 **Package** solves this problem.

1. What is a Package?

A **package** in Java is a way of **grouping related classes, interfaces, and other Java files together**.

Simple-aa:

Package = Folder-like container for Java classes

For example:

MyProject

|

├── Student.java

├── Teacher.java

└─ Car.java

└─ Bank.java

└─ Calculator.java

└─ Employee.java

Everything is mixed together.

Instead, we can organize them:

MyProject

|

└─ students

|

└─ Student.java

|

└─ StudentDetails.java

|

└─ employees

|

└─ Employee.java

|

└─ Manager.java

|

└─ vehicles

|

└─ Car.java

|

└─ Bike.java

|

└─ bank

└─ Account.java

└─ Bank.java

Here:

students

employees

vehicles

bank

are **packages**.

2. Why Do We Need Packages?

Packages are mainly used for:

1. Organization

Large projects contain many classes.

Packages help us organize them.

college.student

college.teacher

college.department

2. Avoiding Name Conflicts

Suppose two different developers create a class called:

Student

Both classes have the same name.

Without packages, there can be confusion.

With packages:

college.Student

school.Student

Now Java can distinguish them.

They are different classes because their **fully qualified names** are different.

college.Student

school.Student

3. Access Control

Packages work together with Java's access modifiers.

Especially:

public

private

protected

default

For example, a member with **default access** can be accessed only within the same package.

We'll study this connection carefully later.

4. Reusability

You can create a useful class inside a package and use it from another Java program.

For example:

```
package calculator;
```

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Another class can import it and use it.

3. Types of Packages

Java packages can broadly be understood as:

1. User-defined packages

Packages created by programmers.

Example:

```
package mypackage;
```

2. Built-in / Java API packages

Packages provided by Java.

Examples:

java.lang

java.util

java.io

java.time

java.net

We'll study these in detail.

4. Creating a Package

Let's create our own package.

Suppose we have:

Student.java

At the top of the file:

package students;

Then:

package students;

```
public class Student {
```

```
    public void display() {
```

```
        System.out.println("Student details");
```

```
    }
```

```
}
```

That's it.

The statement:

package students;

tells Java:

"This Student class belongs to the students package."

5. Important Rule: Package Statement Position

The package statement normally comes at the **beginning of the Java source file**.

Example:

```
package students;
```

```
public class Student {
```

```
}
```

You should not write:

```
public class Student {
```

```
}
```

```
package students;
```

✗ Wrong.

6. Package Name Convention

Java package names are normally written in **lowercase**.

Good:

```
package students;
```

```
package college;
```

```
package banking;
```

```
package com.example.project;
```

Avoid:

```
package Students;
```

```
package College;
```

The lowercase convention is followed to make package names easy to distinguish from class names.

7. Creating a Package with Multiple Classes

One package can contain multiple classes.

Example:

students

|

├── Student.java

├── Teacher.java

└── Course.java

Student.java

```
package students;
```

```
public class Student {
```

```
    public void showStudent() {
```

```
        System.out.println("Student");
```

```
    }
```

```
}
```

Teacher.java

```
package students;
```

```
public class Teacher {
```

```
    public void showTeacher() {
```

```
        System.out.println("Teacher");
```

```
    }
```

```
}
```

Both belong to:

students

because both have:

```
package students;
```

8. Package and Folder Relationship

This is very important.

Usually the package structure corresponds to the directory structure.

For example:

```
package college.students;
```

The corresponding folder structure is:

college

└─ students

So:

college.students

means:

college/students

Think:

. → folder separation

Example:

```
package com.company.project;
```

can correspond to:

com/

company/

project/

9. How to Compile a Package?

Suppose our file is:

Student.java

and contains:

```
package students;
```

```
public class Student {  
  
    public void display() {  
        System.out.println("Hello");  
    }  
}
```

We can compile it using:

```
javac -d . Student.java
```

Here:

```
javac
```

means Java compiler.

```
-d
```

means destination directory.

```
.
```

means current directory.

So:

```
javac -d . Student.java
```

tells Java:

Compile this file and create the package directory structure in the current location.

It may create:

```
students/
```

```
    Student.class
```

10. Using a Class from Another Package

Now the important part.

Suppose we have:

students

Student.java

main

Main.java

Student belongs to:

package students;

Now we want to use Student inside Main.

We use:

import students.Student;

11. What is import?

import tells Java:

"I want to use a class from another package."

Example:

import students.Student;

Now we can write:

Student s = new Student();

instead of writing the full package name every time.

12. Complete Example

Student.java

package students;

public class Student {

```
public void display() {  
    System.out.println("Hello Student");  
}  
}
```

Main.java

```
import students.Student;  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s = new Student();  
  
        s.display();  
    }  
}
```

Output:

Hello Student

13. Understand What Actually Happened

Let's break it down.

Step 1

Student belongs to:

students

because:

package students;

Step 2

Main wants to use Student.

So Main writes:

```
import students.Student;
```

Meaning:

students → package

Student → class

Step 3

Now Java knows which Student class we mean.

So:

```
Student s = new Student();
```

works.

14. package vs import

This is a very common interview question.

package

Used to **declare which package the current class belongs to.**

Example:

```
package students;
```

Meaning:

This class belongs to students package.

import

Used to **use a class from another package.**

Example:

```
import students.Student;
```

Meaning:

I want to use Student from students package.

So remember:

package → where my class belongs

import → which external class I want to use

15. Can We Import the Entire Package?

Yes.

Suppose package contains:

students

└─ Student

└─ Teacher

└─ Course

Instead of:

```
import students.Student;
```

```
import students.Teacher;
```

```
import students.Course;
```

we can write:

```
import students.*;
```

* means:

Import classes available in this package.

Then:

```
Student s = new Student();
```

```
Teacher t = new Teacher();
```

```
Course c = new Course();
```

can be used.

16. Important: * Does NOT Import Subpackages

Suppose:

college

|

|— students

| └─ Student.java

|

└─ teachers

 └─ Teacher.java

If you write:

```
import college.*;
```

it does **not** automatically import:

college.students

college.teachers

Packages and subpackages are treated separately.

This is important.

17. Fully Qualified Class Name

You don't always have to use import.

You can directly use the complete class name.

Example:

```
students.Student s = new students.Student();
```

Here:

students.Student

is called the **fully qualified name**.

It contains:

package + class

Example:

students.Student

18. Import vs Fully Qualified Name

Using import

```
import students.Student;
```

```
Student s = new Student();
```

Without import

```
students.Student s = new students.Student();
```

Both can work.

Usually import makes the code cleaner.

19. Java API Packages

Now we come to the second major topic.

What is Java API?

Java provides thousands of ready-made classes and interfaces.

These are organized into packages.

These ready-made classes are called part of the **Java API (Application Programming Interface)**.

Instead of creating everything ourselves, we can use Java's existing classes.

For example, we need an ArrayList.

Instead of creating our own dynamic array implementation, Java provides:

ArrayList

inside:

java.util

So:

```
import java.util.ArrayList;
```

20. Important Java API Packages

You should know these:

Package	Main purpose
java.lang	Basic Java language classes
java.util	Collections and utility classes
java.io	Input/output
java.nio	Modern file and I/O operations
java.time	Date and time
java.net	Networking
java.sql	Database connectivity
java.math	Mathematical operations
java.text	Formatting and parsing
java.awt	GUI-related classes
javax.swing	GUI components

For beginner/interview preparation, focus especially on:

java.lang

java.util

java.io

java.time

java.net

java.sql

21. java.lang Package

This is one of the most important packages.

It contains fundamental Java classes.

Examples:

String

System

Math

Object

Integer

Double

Boolean

Character

Thread

Example:

```
String name = "Hello";
```

You might wonder:

We never wrote `import java.lang.String`. Why?

Because `java.lang` is **automatically imported** into every Java program.

So:

`String`

works directly.

Similarly:

```
System.out.println("Hello");
```

works without:

```
import java.lang.System;
```

22. java.util Package

This is extremely important for coding interviews.

It contains many utility and collection classes.

Examples:

ArrayList

LinkedList

HashSet

TreeSet

HashMap

Scanner

Arrays

Collections

Example:

```
import java.util.ArrayList;
```

```
ArrayList<Integer> numbers = new ArrayList<>();
```

Another:

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

So whenever you see:

Scanner

remember:

Scanner → java.util

23. java.io

io means:

Input / Output

Used for reading and writing data.

Examples:

File

FileReader

FileWriter

BufferedReader

BufferedWriter

PrintWriter

For example:

```
import java.io.File;
```

or:

```
import java.io.BufferedReader;
```

24. java.time

Used for modern date and time operations.

Examples:

LocalDate

LocalTime

LocalDateTime

Period

Duration

Example:

```
import java.time.LocalDate;
```

```
LocalDate today = LocalDate.now();
```

25. java.net

Used for networking.

Examples include classes for:

URL

URI

Socket

InetAddress

For example:

```
import java.net.URL;
```

Networking applications use these classes.

26. java.sql

Used for database-related operations using JDBC.

Examples:

Connection

Statement

PreparedStatement

ResultSet

DriverManager

Example:

```
import java.sql.Connection;
```

Later, when you study JDBC, this package becomes very important.

27. java.math

Used for mathematical operations requiring special precision or large numbers.

Important classes:

BigInteger

BigDecimal

Example:

```
import java.math.BigInteger;
```

BigInteger can work with integers much larger than the normal int or long limits.

28. Multiple Imports

A Java file can have multiple imports.

Example:

```
import java.util.Scanner;
```

```
import java.util.ArrayList;
```

```
import java.util.HashMap;
```

Then you can use:

Scanner

ArrayList

HashMap

inside your class.

29. Example Using Multiple Packages

```
import java.util.Scanner;
```

```
import java.util.ArrayList;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        ArrayList<Integer> numbers = new ArrayList<>();
```

```
        numbers.add(10);
```

```
        numbers.add(20);
```

```
        System.out.println(numbers);
```

```
    }
```

```
}
```

Here:

Scanner → java.util

ArrayList → java.util

Both come from the same package.

So alternatively:

```
import java.util.*;
```

could be used.

30. Do We Need to Import Every Class?

No.

There are three important situations.

Situation 1: Same package

If two classes are in the same package, you generally don't need an import between them.

Example:

students

├── Student.java

└── Teacher.java

Both:

```
package students;
```

They can use each other without an explicit import.

Situation 2: java.lang

Automatically imported.

So:

String

System

Math

work without imports.

Situation 3: Other packages

Usually you need to import the required class.

Example:

```
import java.util.Scanner;
```

31. Package + Class + Object Connection

You already know objects.

Let's connect everything.

Suppose:

```
package students;
```

```
public class Student {
```

```
    public void study() {
```

```
        System.out.println("Studying");
```

```
    }
```

```
}
```

This tells us:

Package = students

Class = Student

Method = study()

Now another class:

```
import students.Student;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Student s = new Student();
```

```
s.study();  
}  
}
```

Here:

```
import students.Student
```

↓

find Student class

↓

Student s

↓

create object

↓

```
new Student()
```

↓

```
s.study()
```

So **package does not create an object**.

Package only organizes/identifies the class.

Object is still created using:

```
new Student();
```

32. Package Does NOT Mean Object

This distinction is important.

Wrong thinking:

Package → Object

Correct:

Package

↓

contains



Class



object created from class

For example:

students



Student class



Student object

33. Package Naming with Multiple Levels

Packages can have multiple levels.

Example:

```
package com.company.students;
```

Think of it as:

com

└─ company

└─ students

Then a class can be:

```
com.company.students.Student
```

This is the fully qualified class name.

34. Why Do Companies Use com.company?

You may see things like:

```
com.google
```

```
com.microsoft
```

```
com.example
```

org.springframework

This is a common naming convention.

For example:

```
package com.example.myapp;
```

It helps create globally unique package names.

35. Package and Access Modifiers

This is where packages become more powerful.

You learned:

public

private

protected

default

Packages interact with them.

Let's understand the basic idea.

public

Accessible from anywhere, provided the class/member is otherwise accessible.

private

Accessible only inside the same class.

protected

Accessible within the same package and also through inheritance in other packages, subject to Java's access rules.

default

If you don't write an access modifier:

```
int age;
```

that member has **package-private/default access**.

It is accessible within the same package.

36. Example of Default Access

Suppose:

```
package students;
```

```
class Student {  
  
    void display() {  
        System.out.println("Hello");  
    }  
}
```

Because we didn't write:

```
public
```

the class has default/package-private access.

It can be accessed from classes in the same package, but not normally from an unrelated class in another package.

This is one reason packages are useful for **encapsulation and access control**.

37. Package and Encapsulation

You learned encapsulation on Day 16.

Remember:

Encapsulation = controlling access to data/behavior.

Packages provide another level of organization and access control.

For example:

bank

├— Account

├— Transaction

└— BankService

You can keep certain implementation details package-private and expose only the required public classes/methods.

So package organization can support good software design.

38. Common Mistake: Importing the Package Instead of Class

Some beginners write:

```
import java.util;
```

✗ That's not the normal syntax for importing classes.

Instead:

```
import java.util.Scanner;
```

or:

```
import java.util.*;
```

Remember:

```
import package.ClassName;
```

or:

```
import package.*;
```

39. Common Mistake: Writing java.util.Scanner as an Import

Correct:

```
import java.util.Scanner;
```

Then:

```
Scanner sc = new Scanner(System.in);
```

Incorrect:

```
import java.util;
```

40. Common Mistake: Thinking import * Imports Everything

This:

```
import java.util.*;
```

doesn't mean:

Import every Java class from everywhere.

It means:

Make the classes directly in java.util available by simple name.

It does not recursively import subpackages.

41. Important Interview Question

Q: What is a package in Java?

Answer:

A package is a namespace used to group related classes, interfaces, and other types. It helps organize code, avoid naming conflicts, and provide package-level access control.

42. What is import?

import is a Java keyword used to make classes or types from another package available for use by their simple names.

Example:

```
import java.util.Scanner;
```

43. Difference Between Package and Import

Package	Import
Declares where a class belongs	Allows using a class from another package
Written using package	Written using import
Usually at top of source file	After package declaration and before class declaration
Example package college;	Example import java.util.Scanner;

Remember:

package → belongs to

import → use from

44. What is Java API?

Interview answer:

Java API is a collection of predefined classes, interfaces, and packages provided by Java to perform common programming tasks.

Examples:

java.lang

java.util

java.io

java.time

java.net

java.sql

45. Most Important Java API Classes to Remember

For your current Java preparation, remember this table:

Class	Package
String	java.lang
System	java.lang
Math	java.lang
Integer	java.lang
Scanner	java.util
ArrayList	java.util
LinkedList	java.util
HashMap	java.util
HashSet	java.util
Arrays	java.util
Collections	java.util

Class	Package
File	java.io
BufferedReader	java.io
LocalDate	java.time
LocalDateTime	java.time
BigInteger	java.math
Connection	java.sql

46. One Very Important Concept: Namespace

You may hear the word **namespace** in interviews.

A namespace is basically a way to provide a unique name/context for things.

Suppose we have:

college.Student

school.Student

Both classes are named:

Student

But their packages are different.

Therefore:

college.Student

and

school.Student

are different fully qualified names.

Packages therefore act as **namespaces**.

47. Real-World Example

Imagine a college.

There are different departments:

College

|

|— CSE

|— ECE

|— EEE

└— MECH

Each department may have a:

Student

So:

CSE.Student

ECE.Student

EEE.Student

MECH.Student

Same class name:

Student

Different package/namespace:

CSE

ECE

EEE

MECH

That's the basic idea behind packages.

48. Complete Mental Model □

Remember this hierarchy:

Java Project

↓

Packages

↓

Classes / Interfaces



Objects

For example:

MyProject



students



Student



new Student()



Student object

And when another package wants to use it:

other package



import students.Student



Student



new Student()

49. package, import, class, object

Let's clearly differentiate all four.

Package

Organizes classes.

package students;

Import

Allows access to a class from another package by simple name.

```
import students.Student;
```

Class

Blueprint.

```
class Student {  
  
}
```

Object

Actual instance.

```
Student s = new Student();
```

So:

Package → organizes

Import → brings class into scope

Class → blueprint

Object → instance

50. What You Should Remember for Interviews

Packages

- Package groups related classes.
- Package helps organize large projects.
- Package helps avoid naming conflicts.
- Package provides package-level access control.
- Package is declared using package.
- Package statement is normally at the top.
- Package names are conventionally lowercase.
- package a.b.c; represents a hierarchical package name.

Import

- import allows us to refer to types from another package by simple name.
- Example:

```
import java.util.Scanner;
```

- Wildcard:

```
import java.util.*;
```

- Wildcard does not import subpackages.
- java.lang is automatically imported.

Java API

Know these:

java.lang → basic language classes

java.util → collections/utilities

java.io → input/output

java.time → date/time

java.net → networking

java.sql → database

java.math → big/specialized numbers

51. Super Simple Revision

If you forget everything from today's lesson, remember this:

PACKAGE



Organizes related classes

```
package students;
```



IMPORT



Use class from another package

```
import students.Student;
```



CLASS



Blueprint

Student



OBJECT



Actual instance

```
Student s = new Student();
```

And:

java.lang → automatically available

java.util → Scanner, ArrayList, HashMap...

java.io → File, BufferedReader...

java.time → LocalDate, LocalTime...

java.net → networking

java.sql → database

One-line memory trick

Package = where the class belongs; Import = which external class I want to use.

That is the core of **Day 18: Packages**.

